

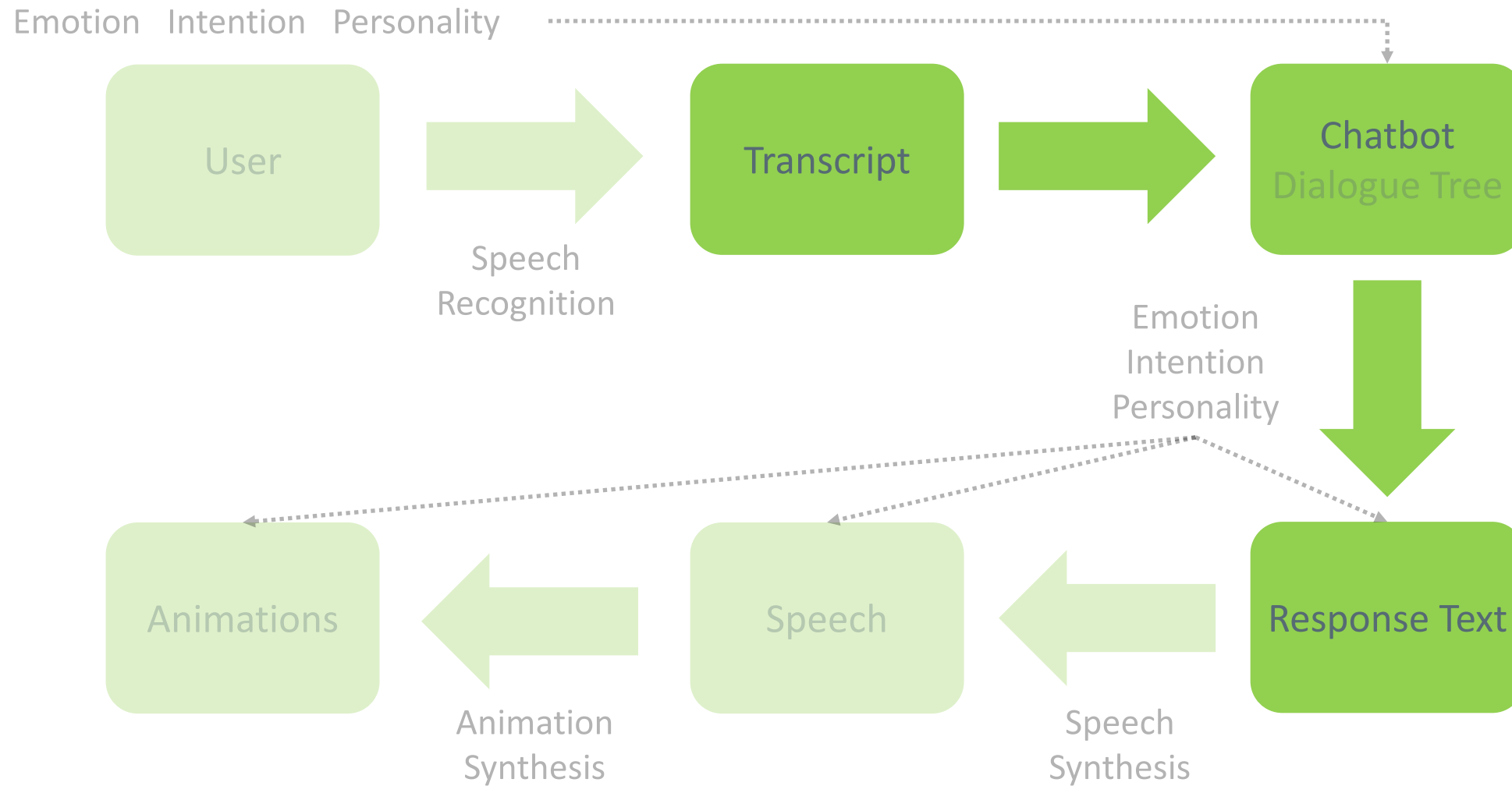


Artificial Intelligence for Digital Characters

Chatbot II

Dr. Rafael Wampfler, ETH Zürich

Conversational Digital Character



Prompting

Retrieval Augmented Generation
Evaluation of Chatbots

In-Context Learning
Few-Shot Learning
Prompt Optimizations
Finetuning

Prompting: System Prompts & Role Assignment

Chat models distinguish three message roles that shape the entire conversation.

system

Sets the AI persona, constraints, and behavior before the conversation starts.
Runs once; invisible to the end-user.

user

The human input: questions, tasks or instructions sent each turn.
Main input the model responds to.

assistant

The model's own previous replies.
Pre-fill to steer or to simulate a conversation history.

Example: Persona + Constraint System Prompt

```
messages = [  
  {"role": "system",    "content": "You are Einstein, a friendly physicist.  
    Answer only physics questions. Be concise."},  
  {"role": "user",      "content": "What is E=mc squared?"},  
  {"role": "assistant", "content": "Energy equals mass times c squared..."},  
  {"role": "user",      "content": "Tell me a joke."},  
]
```

Key Uses of System Prompts

Persona / Role -- Give model a character or domain expertise

Constraints -- Restrict topics, language, length or output format

Context -- Inject background knowledge or retrieved documents (RAG)

Safety guardrails -- Define ethical boundaries and refusal behaviour

Prompting: In-Context Learning & Instructions

- **In-context learning:** Improving model performance without gradient-based updates
- **Instruction tuning:** Fine-tuning on instruction-response pairs to help LLMs to correctly respond to instructions.

Hotel Review Completions

Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relaxed. In short our stay was

... not a pleasant one. The staff at the front desk were not welcoming or friendly, and seemed disinterested in providing good customer service.

... uncomfortable and not worth the price we paid. We will not be returning to this hotel.

Prompt

Responses

Prompting: Style of Prompt

A prompt consisting of a review plus an incomplete statement

Human: Do you think that “input” has negative or positive sentiment?

Choices:

(P) Positive

(N) Negative

Assistant: I believe the best answer is: (

Instruction at the end helps with constraining the generation

Prompting: Style of Prompt

LLM Outputs for Basic Prompts

Original Review (\$INPUT)	Did not like the service that I was provided, when I entered the hotel. I also did not like the area, in which the hotel was located. Too much noise and events going on for me to feel relax and away from the city life.
Sentiment	Prompt: \$INPUT + In short, our stay was Output: not enjoyable
Fine-grained Sentiment	Prompt: \$INPUT + These aspects were important to the reviewer: Output: 1. Poor service 2. Unpleasant location 3. Noisy and busy area
Summarization	Prompt: \$INPUT + tl;dr Output: I had a bad experience with the hotel's service and the location was loud and busy.
Translation	Prompt: \$INPUT + Translate this to French Output: Je n'ai pas aimé le service qui m'a été offert lorsque je suis entré dans l'hôtel. Je n'ai également pas aimé la zone dans laquelle se trouvait l'hôtel. Trop de bruit et d'événements pour que je me sente détendu et loin de la vie citadine.

Prompting: Structured Outputs & Function Calling

① Structured Output

Constrain the LLM to respond in a defined schema (e.g. JSON).

```
from openai import OpenAI
from pydantic import BaseModel
from typing import Literal

class MovieReview(BaseModel):
    sentiment: Literal["positive", "negative"] =
        Field(description="Overall sentiment
            of the review")
    score: float = Field(ge=0.0, le=1.0,
        description="Confidence
            score between 0 and 1")

client = OpenAI()
response = client.beta.chat.completions.parse(
    model = "gpt-4o",
    messages = [{"role": "user",
        "content": "Review: Great film!"}],
    response_format = MovieReview,
)

# Guaranteed output:
# { "sentiment": "positive",
#   "score": 0.92 }
```

② Function Calling (Tool Use)

LLM decides when & how to call an external function.

```
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current weather",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string"}
            }, "required": ["city"]
        }
    }
}]

resp = client.chat.completions.create(
    model = "gpt-4o",
    messages = [{"role": "user",
        "content": "Weather in Zurich?"}],
    tools = tools,
)

# LLM returns a tool call:
# { "name": "get_weather",
#   "arguments": {"city": "Zurich"} }
```

Structured outputs guarantee valid JSON; function calling lets the LLM invoke real-world APIs and return structured arguments.

Prompting: ReAct

ReAct (Yao et al., 2022) interleaves reasoning traces with tool actions, letting LLMs plan, call APIs, and update their plan based on results.

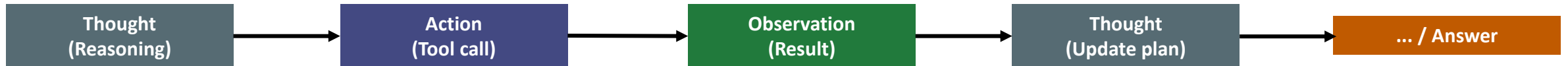
Chain-of-Thought only

```
Thought: Einstein was born in 1879.  
Thought: He died in 1955.  
Thought:  $1955 - 1879 = 76$ .  
Answer: 76 years old.
```

Problem: Model may hallucinate
facts with no grounding.

ReAct (Thought + Action + Observation)

```
Thought: I need Einstein's birth year.  
Action: search('Einstein birth year')  
Observation: '1879 in Ulm, Germany'  
Thought: I need his death year.  
Action: search('Einstein death year')  
Observation: '1955 in Princeton, NJ'  
Thought:  $1955 - 1879 = 76$ .  
Answer: 76 years old.
```



Key Advantages

Reduces hallucination by grounding reasoning in retrieved facts

Human-interpretable trace of reasoning steps

Foundation of modern LLM agents (LangChain, LlamaIndex, OpenAI Agents SDK)

Definition: This task is about writing a correct answer for the reading comprehension task. Based on the information provided in a given passage, you should identify the shortest continuous text span from the passage that serves as an answer to the given question. Avoid answers that are incorrect or provides incomplete justification for the question.

Passage: Beyoncé Giselle Knowles-Carter (born September 4, 1981) is an American singer, songwriter, record producer and actress. Born and raised in Houston, Texas, she performed in various singing and dancing competitions as a child, and rose to fame in the late 1990s as lead singer of R&B girl-group Destiny's Child. Managed by her father, Mathew Knowles, the group became one of the world's best-selling girl groups of all time. Their hiatus saw the release of Beyoncé's debut album, *Dangerously in Love* (2003), which established her as a solo artist worldwide, earned five Grammy Awards and featured the Billboard Hot 100 number-one singles "Crazy in Love" and "Baby Boy".

Examples:

Q: In what city and state did Beyoncé grow up?

A: Houston, Texas

Q: What areas did Beyoncé compete in when she was growing up?

A: singing and dancing

Q: When did Beyoncé release *Dangerously in Love*?

A: 2003

Q: When did Beyoncé start becoming popular?

A:

Choosing set of examples that most increases task performance (e.g., classification accuracy) of the prompt on a test set.

DSPy (<https://github.com/stanfordnlp/dspy>)
Automatically find optimum set of examples

Few-Shot Learning: DSPy

Task: Classify text into positive and negative

Metric: Accuracy

```
trainset = [  
  ("I love this product!", "positive"),  
  ("This is amazing", "positive"),  
  ("Terrible experience", "negative"),  
  ("I hate it", "negative"),  
  ("Not bad actually", "positive"),  
  ("Worst purchase ever", "negative"),  
]
```

Teacher model: strong LLM
with no few-shots

Only keep correct
candiddates

```
trainset = [  
  ("I love this product!", "positive"),  
  ("Terrible experience", "negative"),  
  ("Not bad actually", "positive"),  
]
```

Random search: Try many random combinations, keep the best-performing

Other algorithms available (bayesian optimization)

Prompting for a Variant

Generate a variation of the following instruction while keeping the semantic meaning.

Input: {INSTRUCTION}

Output: {COMPLETE}

Given access to labeled training data, candidate prompts can be scored based on task performance (e.g., accuracy)

Automatic Prompt Optimization

1. Run the prompt on a sample of training examples,
2. Identify examples where the prompt fails,
3. Ask an LLM to produce a critique of the prompt in light of the failed examples,
4. Provide the resulting critique to an LLM, and ask it to generate improved prompts.

Critiquing Prompt

I'm trying to write a zero-shot classifier prompt.
My current prompt is: {prompt}
But this prompt gets the following examples wrong:
{error_string}
Give {num_feedbacks} reasons why the prompt could have gotten these examples wrong.

Prompt Improvement Prompt

I'm trying to write a zero-shot classifier. My current prompt is:
{prompt}
But it gets the following examples wrong: {error_str}

Based on these examples the problem with this prompt is that {gradient}.
Based on the above information, I wrote {steps_per_gradient} different improved prompts. Each prompt is wrapped with <START> and <END>.

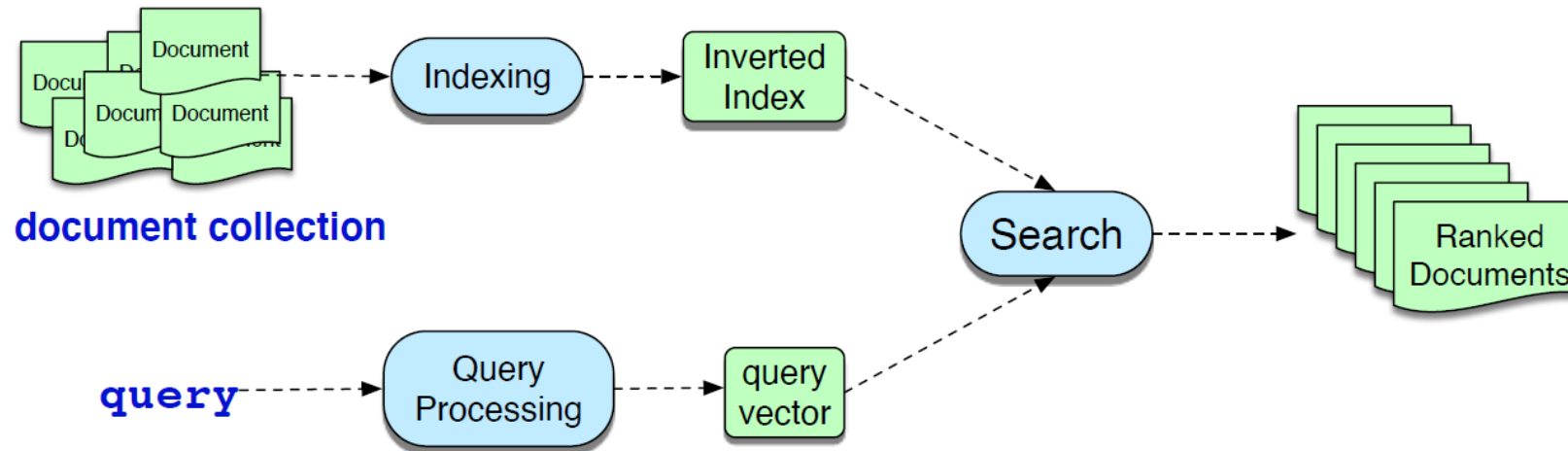
The {steps_per_gradient} new prompts are:

Prompting
RAG
Evaluation of Chatbots

Information Retrieval
RAG

Prompting has two problems

- LLM can hallucinate
- No access to proprietary or new data



Chunking Strategies for RAG

Before indexing, documents must be split into chunks. Chunk size and strategy directly affect retrieval precision and LLM context quality.

Fixed-Size

Split every N tokens with optional overlap.
Fast and simple; may break sentences mid-thought.
Best practice: 10-20% overlap (e.g. 100 tokens overlap on 500-token chunks).

Recursive

Iteratively split by paragraph -> sentence
-> word until target size is reached.
Keeps semantic units intact.
Default in LangChain.

Semantic

Use sentence embeddings to detect topic shifts and split at semantic boundaries.
Higher quality; computationally expensive.
Best for heterogeneous documents.

Fixed-Size with Overlap (Code)

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size = 500,    # tokens per chunk
    chunk_overlap = 100, # 20% overlap
    separators = ['\n\n', '\n', ' ', ''], # priority order
)
chunks = splitter.split_text(document_text)
```

Choosing Chunk Size

Small chunks (100-200 tokens) -- Higher precision; risk of missing cross-sentence context

Large chunks (500-1000 tokens) -- More context; risk of exceeding LLM context window or diluting relevance

Structure-aware chunking -- Respect document sections, headings, tables for best quality

Rule of thumb: chunk size should fit comfortably within the embedding model's token limit (typically 512 tokens).

- Computing a term weight for each word (t) in a document (d)
- Tf-idf: term frequency (tf), inverse document frequency (idf)

$$\text{tf}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t, d) & \text{if } \text{count}(t, d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\text{idf}_t = \log_{10} \frac{N}{\text{df}_t}$$

N : Number of documents

df_t : Number of documents in which term t occurs

$$\text{tf-idf}(t, d) = \text{tf}_{t,d} \cdot \text{idf}_t$$

Scoring document d with the query q

Example:

Query: sweet love
Doc 1: Sweet sweet nurse! Love?
Doc 2: Sweet sorrow
Doc 3: How sweet is love?
Doc 4: Nurse!

$$\text{score}(q, d) = \sum_{t \in \mathbf{q}} \frac{\text{tf-idf}(t, q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i, q)}} \cdot \frac{\text{tf-idf}(t, d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i, d)}}$$

$$\text{score}(q,d) = \sum_{t \in q} \overbrace{\log \left(\frac{N}{df_t} \right)}^{\text{IDF}} \overbrace{\frac{tf_{t,d}}{k \left(1 - b + b \left(\frac{|d|}{|d_{\text{avg}}|} \right) \right) + tf_{t,d}}}^{\text{weighted tf}}$$

$|d_{\text{avg}}|$: length of average document

k : balance between tf and idf, typically $k \in [1.2, 2]$

b : controls importance of document length normalization
(1 = scaling by document length, 0 = no length scaling)
typically $b = 0.75$

- Data structure to efficiently find documents that contain words in the query
 - Input: Query word
 - Output: List of documents containing the term
- Inverted index consists of dictionary and postings
 - Dictionary: List of words, each pointing to a postings list (plus document frequency for each word)
 - Postings list: List of document Ids associated with each term (plus word frequency)

Query: sweet love

Doc 1: Sweet sweet nurse! Love?

Doc 2: Sweet sorrow

Doc 3: How sweet is love?

Doc 4: Nurse!

how	{1}	→	3	[1]		
is	{1}	→	3	[1]		
love	{2}	→	1	[1] → 3	[1]	
nurse	{2}	→	1	[1] → 4	[1]	
sorry	{1}	→	2	[1]		
sweet	{3}	→	1	[2] → 2	[1] → 3	[1]

- Tf-idf and BM25: They only work when the query and document share words
- Bi-encoder
 - Each document is encoded in advance using text embedder (e.g., SBERT, OpenAI)
 - Query is encoded and then dot product between query embedding and document embeddings calculated
 - Documents with highest dot product are selected
- ColBERT
$$\text{score}(q, d) = \sum_{i=1}^N \max_{j=1}^m \mathbf{E}_{q_i} \cdot \mathbf{E}_{d_j}$$
 - Embedding for each query and document token
 - MaxSim: Most similar token in document d for a query token q_i
 - Document score $\text{score}(q, d)$: Summing MaxSim scores over all query tokens

- Why vector databases?
 - Calculating dot products against millions of documents computationally impossible
 - Solution: Approximate Nearest Neighbor algorithms to find similar vectors efficiently (e.g., FAISS, Pinecone, Milvus, Weaviate)
 - Trade-off: Slight accuracy loss for massive speed gains
- How vector databases work
 - Indexing: Organize embeddings in specialized data structures
 - Query processing: Convert query to embedding vector
 - Fast retrieval: ANN algorithms find top-k similar vectors in ms

Reranking in RAG

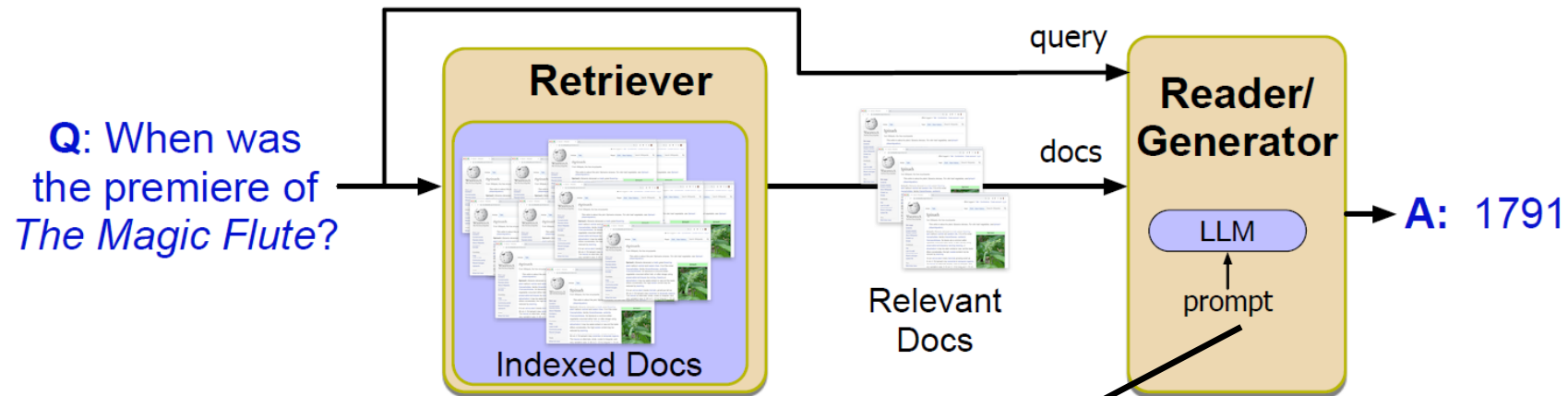
After fast first-stage retrieval (bi-encoder / BM25), a cross-encoder reranker rescores the top-k candidates for higher precision.



	Bi-encoder	Cross-encoder
How it works	Encodes query & docs separately; dot product	Encodes query+doc jointly; full attention across both
Speed	Very fast (pre-computed)	Slow (per query-doc pair)
Accuracy	Good recall	High precision
Use in RAG	Stage 1: retrieve top-100	Stage 2: rerank to top-5
Example model	SBERT, OpenAI ada	ms-marco-MiniLM-L-6-v2

```
from sentence_transformers import CrossEncoder
model = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
scores = model.predict([(query, doc) for doc in candidates]) # Stage 2
```

Retrieval Augmented Generation



Schematic of a RAG Prompt

retrieved passage 1

retrieved passage 2

...

retrieved passage n

Based on these texts, answer this question: Q: Who wrote the book "The Origin of Species"? A:

- Precision@k
 - Proportion of top-k retrieved documents that are relevant
 - $\text{Precision@k} = (\# \text{ relevant in top-k}) / k$
 - Example: 3 relevant in top-5 $\rightarrow \text{Precision@5} = 0.6$
- Recall@k
 - Proportion of relevant documents found in top-k
 - $\text{Recall@k} = (\# \text{ relevant in top-k}) / (\text{total relevant})$
 - Example: 3 relevant in top-5 and 8 total relevant $\rightarrow \text{Recall@5} = 0.375$
- Mean Reciprocal Rank (MRR)
 - Average of the reciprocal ranks of the first correct answer for a set of N queries
 - $MRR = \left(\frac{1}{N}\right) \sum_{i=1}^N \frac{1}{\text{rank}_i}$
 - First relevant at position 2 (N = 1) $\rightarrow MRR = 0.5$

- Grounding
 - Is the generated answer strictly derived from retrieved documents?
 - Use LLM to check each claim in answer against retrieved documents
 - From 0.0 (hallucinated information) to 1.0 (all claims fully supported)
- Answer relevance
 - Does the answer address the user's original question?
 - Compare embeddings of answer and question
 - Example: What is the capital of France?
 - ✓ The capital of France is Paris.
 - ✗ France has many beautiful cities.

Critical for RAG: These metrics measure if hallucination is prevented and if the system meets user intent

Prompting
Retrieval Augmented Generation

Evaluation of Chatbots

Qualitative Evaluation
Perplexity
BLEU Score
Semantic Similarity

- Mean Opinion Score

Human raters provide a score for each response on a scale (e.g., 1 to 5) based on relevance, fluency, and coherence.

- A/B Testing

Human evaluators compare responses from different versions of the chatbot to determine which version performs better.

- **Perplexity**

Measure how well a probability distribution or model predicts a sample. Lower perplexity indicates a better predictive model.

- **BLEU Score**

BLEU (Bilingual Evaluation Understudy) score is used to evaluate the quality of text which has been machine-translated from one natural language to another. It can be adapted to evaluate the chatbot responses by comparing the chatbot response to a set of reference responses.

- **Semantic Similarity**

Using cosine similarity, word embeddings, or transformer-based models like BERT, to measure the semantic similarity between the chatbot's response and the reference response.

- Perplexity as a metric for language model performance
- Measures uncertainty in predicting the next word
- Lower perplexity indicates better performance
- Commonly used in evaluating language models

- Definition: Perplexity P of a model on a response
- Formula:

$$P = 2^{-\frac{1}{N} \sum_{i=1}^N \log_2 P(w_i | w_1, w_2, \dots, w_{i-1})}$$

- N is the number of words in the response
- $P(w_i | w_1, w_2, \dots, w_{i-1})$ is the predicted probability of the i -th word

- Generated response: *The cat sat on the mat*
- Probabilities:
 - $P(\text{The} | \text{start}) = 0.5$
 - $P(\text{cat} | \text{The}) = 0.2$
 - $P(\text{sat} | \text{The cat}) = 0.1$
 - $P(\text{on} | \text{The cat sat}) = 0.1$
 - $P(\text{the} | \text{The cat sat on}) = 0.3$
 - $P(\text{mat} | \text{The cat sat on the}) = 0.4$
- $P = 2^{-\frac{1}{6}(\log_2 0.5 + \log_2 0.2 + \log_2 0.1 + \log_2 0.1 + \log_2 0.3 + \log_2 0.4)} = 3.81$

- Not always correlates with human judgment
- Sensitive to test set and model training data
- May not capture contextual relevance
- Complements other evaluation metrics

- BLEU Score compares n-grams of the generated text with those of reference texts
- It considers both precision and length penalty
 - Precision: the proportion of n-grams in the generated text that appear in the reference text
 - Length penalty discourages very short responses
- Typically calculated for n-grams from 1 to 4

Formula: $BLEU = BP * \exp(\sum_{n=1}^N w_n \log p_n)$

- BP: Brevity Penalty, accounts for length of generated text (c) relative to reference (r)

$$Brevity\ Penalty = \begin{cases} 1, & \text{if } c > r \\ e^{(1-r/c)}, & \text{if } c \leq r \end{cases}$$

- p_n : Precision for n-grams.
- w_n : Weights, often set equally for simplicity
- Typically N = 4

Quantitative Evaluation: BLEU

- Target sentence: The guard arrived late because it was raining
- Generated sentence: The guard arrived late because of the rain

Target Sentence: The guard arrived late because it was raining
Predicted Sentence: The guard arrived late because of the rain



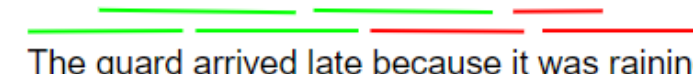
$$\text{Precision 1-gram } (p_1) = 5/8$$

Target Sentence: The guard arrived late because it was raining
Predicted Sentence: The guard arrived late because of the rain



$$\text{Precision 3-gram } (p_3) = 3/6$$

Target Sentence: The guard arrived late because it was raining
Predicted Sentence: The guard arrived late because of the rain



$$\text{Precision 2-gram } (p_2) = 4/7$$

Target Sentence: The guard arrived late because it was raining
Predicted Sentence: The guard arrived late because of the rain



$$\text{Precision 4-gram } (p_4) = 2/5$$

- Target sentence: The guard arrived late because it was raining
- Generated sentence: The guard arrived late because of the rain

Precision 1-gram (p_1) = 5/8

Precision 2-gram (p_2) = 4/7

Precision 3-gram (p_3) = 3/6

Precision 4-gram (p_4) = 2/5

$w_n = 1$

$BP = 1$ ($c = 8, r = 8$)

$$BLEU = BP * \exp\left(\sum_{n=1}^N w_n \log p_n\right) = 0.32$$

- Evaluate different versions of chatbot models
- Compare responses to a set of reference answers
- Identify areas for improvement based on score
- Limitations
 - Does not account for semantic meaning
 - Can be insensitive to word order and context
 - May require multiple reference texts for accuracy

- Semantic similarity measures the closeness in meaning between two pieces of text
- **Vector Space Models:** Use word embeddings like Word2Vec, GloVe
- **Transformer Models:** Leverage models like BERT, RoBERTa for contextual embeddings
- **Cosine Similarity:** Commonly used method to measure the angle between two vectors in a multi-dimensional space

$$\text{similarity} = \cos(\theta) = (A * B) / (|A| |B|)$$

1. Generate response from chatbot
2. Identify a set of reference responses
3. Calculate similarity score between chatbot response and each reference response

Use strong LLMs to automatically evaluate chatbot responses (GPT-5.4, Claude, Gemini)

User: What's the best programming language for beginners?

Response: Python is excellent for beginners due to simple syntax. JavaScript is also good for web development.

Prompt: Rate this response (1-5) on relevance, fluency, and helpfulness. Provide scores and brief reasoning.

Relevance: 5/5

Fluency: 5/5

Helpfulness: 4/5

Directly addresses the question with clear, actionable advice. Could elaborate more on why.

LLM Benchmarks & Leaderboards

Automated benchmarks and human-preference leaderboards complement per-task metrics for holistic chatbot evaluation.

Chatbot Arena

Users compare two anonymous models side-by-side.

6M+ votes; covers text, vision, video. Good for real-world preference signal. Bias: crowd-sourced; gameable.

MMLU

57 multiple-choice subject areas (math, law, medicine, history...). Tests broad factual knowledge.

Standard academic benchmark. Limitation: multiple-choice only; not conversational.

Stanford HELM

Scenario-based, multidimensional: accuracy, calibration, robustness, fairness, efficiency.

Most rigorous; detailed per-model reports. Best for deployment decisions.

When to Use Which

- Chatbot Arena -- Quick signal: how do users prefer your model vs. competitors in the wild?
- MMLU -- Does the model have the factual knowledge needed for your domain?
- Stanford HELM -- High-stakes deployment: need holistic accuracy + safety + fairness scores

Recommendation: Use Chatbot Arena for discovery, task benchmarks (MMLU, HELM) for accountability, and task-specific evaluation (BLEU, Semantic Similarity) for your specific chatbot use case.

Top Models on Chatbot Arena (2025)

Google Gemini (~1385 Elo) > OpenAI GPT (~1366) > DeepSeek (~1362) > xAI Grok (~1288) [Stanford AI Index 2025]

Prompting

- The style of the prompt depends on the task
- Few-shot learning can improve task performance
- Prompt can be improved using LLMs

Retrieval Augmented Generation

- A score between a query and a document can be calculated with tf-idf or BM25
- Documents and queries can be represented as dense vectors (embeddings)
- The inverted index allows for efficient finding of documents for words
- RAG systems are based on retrievers and readers

Evaluation of TTS and Chatbot

- Qualitative evaluation
- Quantitative evaluation

- **Jurafsky Dan and Martin, James H. "Speech and Language Processing (3rd ed. draft)" Chapter 12 & 14**
<https://web.stanford.edu/~jurafsky/slp3/>
- Khattab, O., A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. ICLR.
- Brown, T., B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. 2020. Language models are few-shot learners. NeurIPS, volume 33.
- Honovich, O., U. Shaham, S. R. Bowman, and O. Levy. 2023. Instruction induction: From few examples to natural language task descriptions. ACL.
- Min, S., X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer. 2022. Rethinking the role of demonstrations: What makes in-context learning work? EMNLP.
- Zhou, Y., A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba. 2023. Large language models are human-level prompt engineers. The Eleventh International Conference on Learning Representations.
- Chen, D., A. Fisch, J. Weston, and A. Bordes. 2017. Reading Wikipedia to answer open-domain questions. ACL.
- Izacard, G., P. Lewis, M. Lomeli, L. Hosseini, F. Petroni, T. Schick, J. Dwivedi-Yu, A. Joulin, S. Riedel, and E. Grave. 2022. Few-shot learning with retrieval augmented language models. ArXiv preprint.
- Kamphuis, C., A. P. de Vries, L. Boytsov, and J. Lin. 2020. Which BM25 do you mean? a large-scale reproducibility study of scoring variants. European Conference on Information Retrieval.

- Karpukhin, V., B. O'guz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih. 2020. Dense passage retrieval for open-domain question answering. EMNLP.
- Khattab, O. and M. Zaharia. 2020. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. SIGIR.
- Manning, C. D., P. Raghavan, and H. Schütze. 2008. Introduction to Information Retrieval. Cambridge.
- Radford, A., J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. 2019. Language models are unsupervised multitask learners. OpenAI tech report.
- Ram, O., Y. Levine, I. Dalmedigos, D. Muhlgay, A. Shashua, K. Leyton-Brown, and Y. Shoham. 2023. In-context retrieval-augmented language models. ArXiv preprint.